



Comp90042

Natural Language Processing

Workshop (Week 7 | 2026s1)

Dr Jean Lee





Agenda

1. Discussion (~25 mins)

- **Contextual representations**
- **Recap – Transformer**
- **Large Language Model, GPT series**
- **Hugging Face**

2. Programming (~30 mins)

20th April	7	 Workshop-07.pdf ↓	10-hugging-face.ipynb ↓	
------------	---	---	---	--

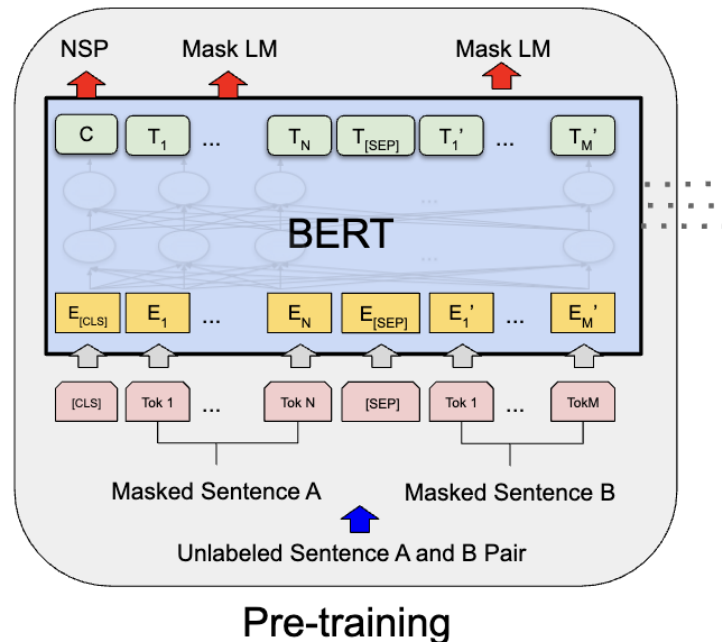
Discussion: Contextual representations?

Discussion

1. What are **contextual representations**?

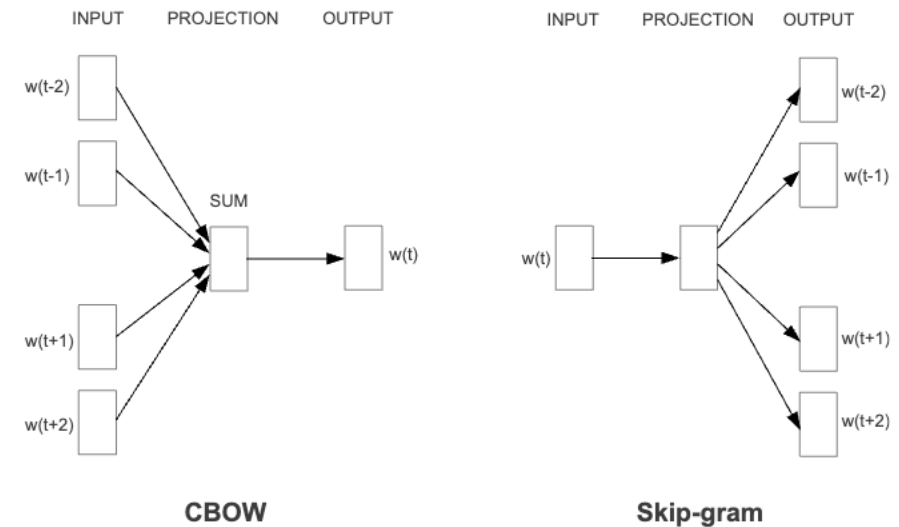
Context Representations (e.g. BERT)

- *Vectors for words in context*



Static Representations (e.g. Word2Vec)

- *Lookup tables, variation of Bag-of-Words*



(**Word2Vec**) Efficient Estimation of Word Representations in Vector Space (ICLR 2013) [[paper](#)]
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding (ACL 2019) [[paper](#)]

Discussion: Contextual representations?

Discussion

1. What are **contextual representations**?

Context Representations (e.g. BERT)

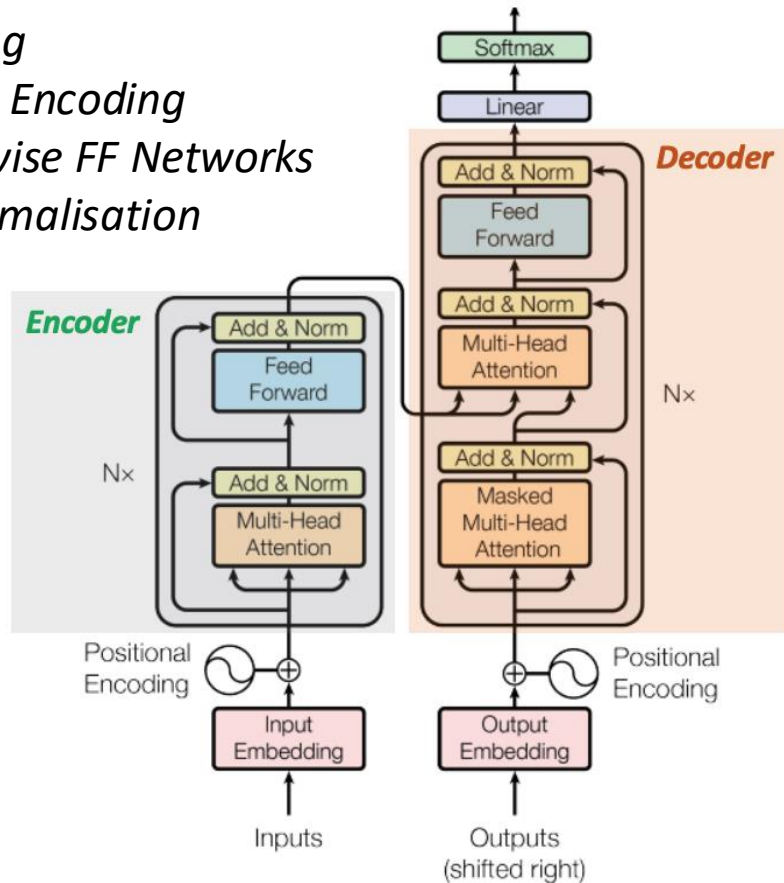
- a representation of the word based on a **particular usage**.
- It captures the **different senses or nuances** of the word depending on the **context**.
- **pre-trained on large data** : obtained fairly comprehensive knowledge about the language.
- The **same word receives different embeddings** in different sentences.

Static Representations (e.g. Word2Vec)

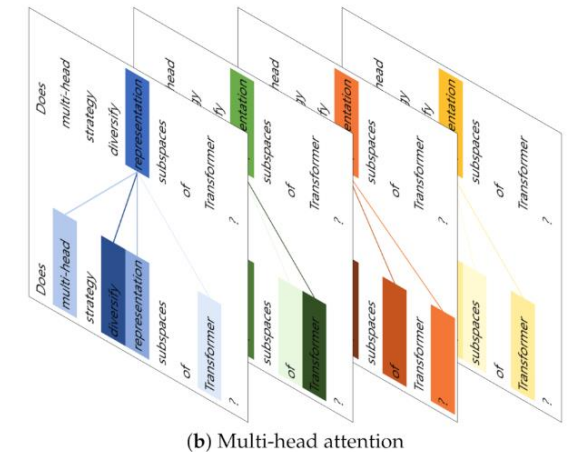
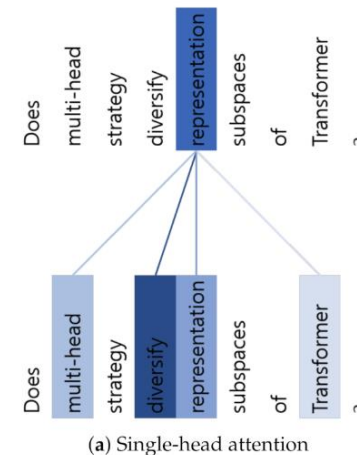
- a single representation for every **word type** (vocabulary entries).
- Static, fixed-dimensional vectors **assigned to each word, regardless of context**.
- (e.g.) “bank” (financial institution) and “bank” (river edge) share the same representation.

Recap : Transformer

- Encoder-Decoder Structure
- **Attention**
- Embedding
- Positional Encoding
- Position-wise FF Networks
- Layer Normalisation



- Transformer uses **attention** to capture dependencies between words.
- For each **target word (Query vector)** in a sentence, transformer **attends to every** other words (**Key vectors**) in the sentence to create its **contextual embeddings**.



Recap : Transformer-code (Decoder-Only)

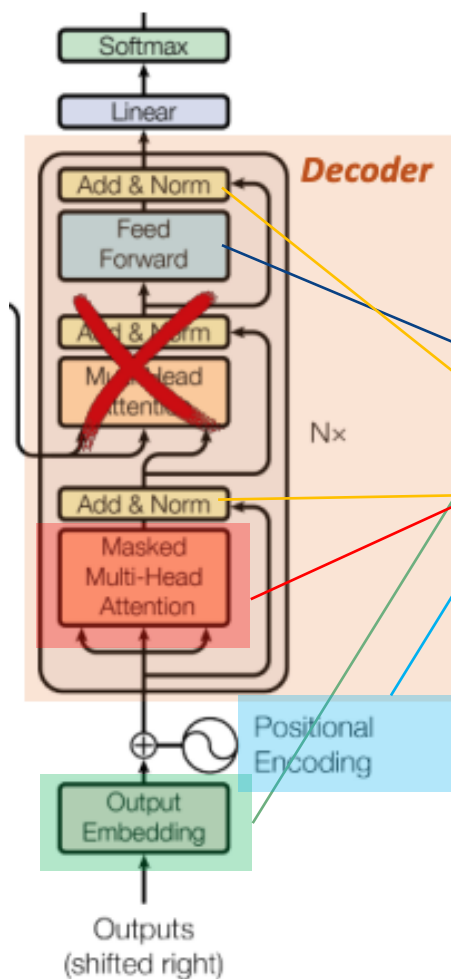


Table of contents

- Transformer with PyTorch
- Loading dataset
- Building vocabulary set
- Transformer Model Archite...
- Embeddings
- Embeddings visualisation
- Positional Encoding
- Position-wise Feed-Forw...
- Attention
- Decoder architecture
- Masking visualisation
- The Complete Vanilla Tra...
- Dataloader pipeline
- Model summary
- Training and Testing
- Sample data processing
- Helper functions

+ Section

3. COMPONENT DIMENSIONS

Embedding layer:

Input: Tokens with vocabulary size 1,813
Output: Vectors of dimension 32

Multi-Head Attention:

Input: Sequence of vectors with dimension 32
Q, K, V projections: 4 heads $\times$ 8 dimensions = 32
Output: Sequence of vectors with dimension 32

Position-wise Feed-Forward Network:

Input dimension: 32
Hidden dimension: 64
Output dimension: 32

Output Classifier:

Input dimension: 32
Output dimension: 1 (sigmoid applied for binary classification)

4. SAMPLE DATA FLOW

Sample: 'Wow... Loved this place.'

Tokenization:

Tokens: ['wow', '...', 'loved', 'this', 'place', '.']
Token IDs: [3, 4, 5, 6, 7, 8]
After truncation/padding (max_len=20): [3, 4, 5, 6, 7, 8, 2, 2, 2, 2]...
Sequence length: 20

Shape Transformations:

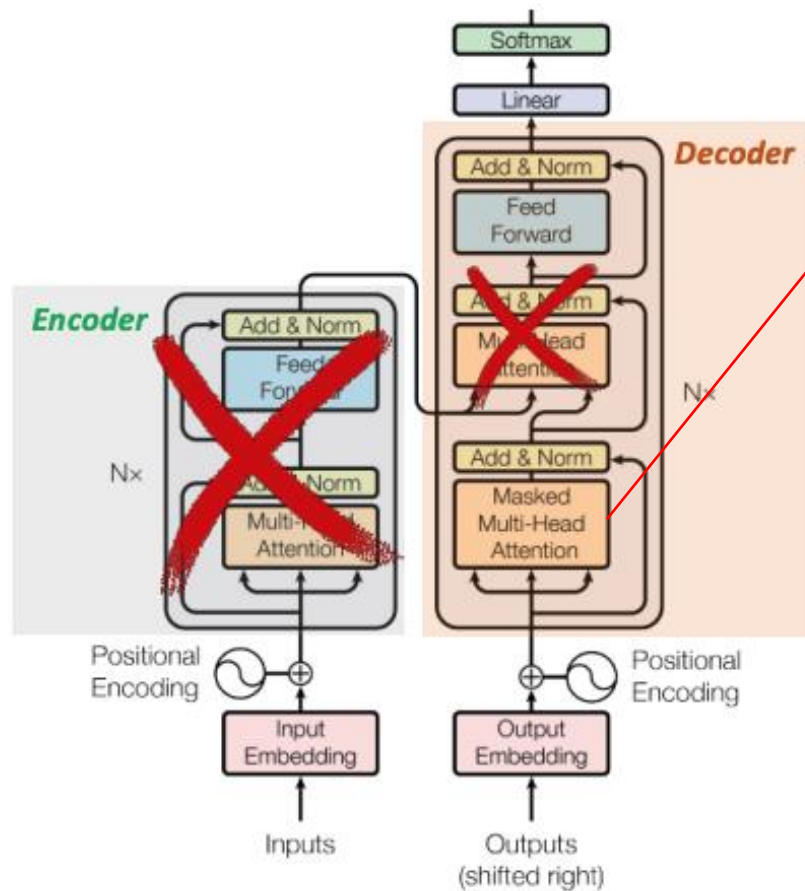
Input token IDs: torch.Size([1, 20])
Attention mask: torch.Size([1, 20])

Embedding Layer:

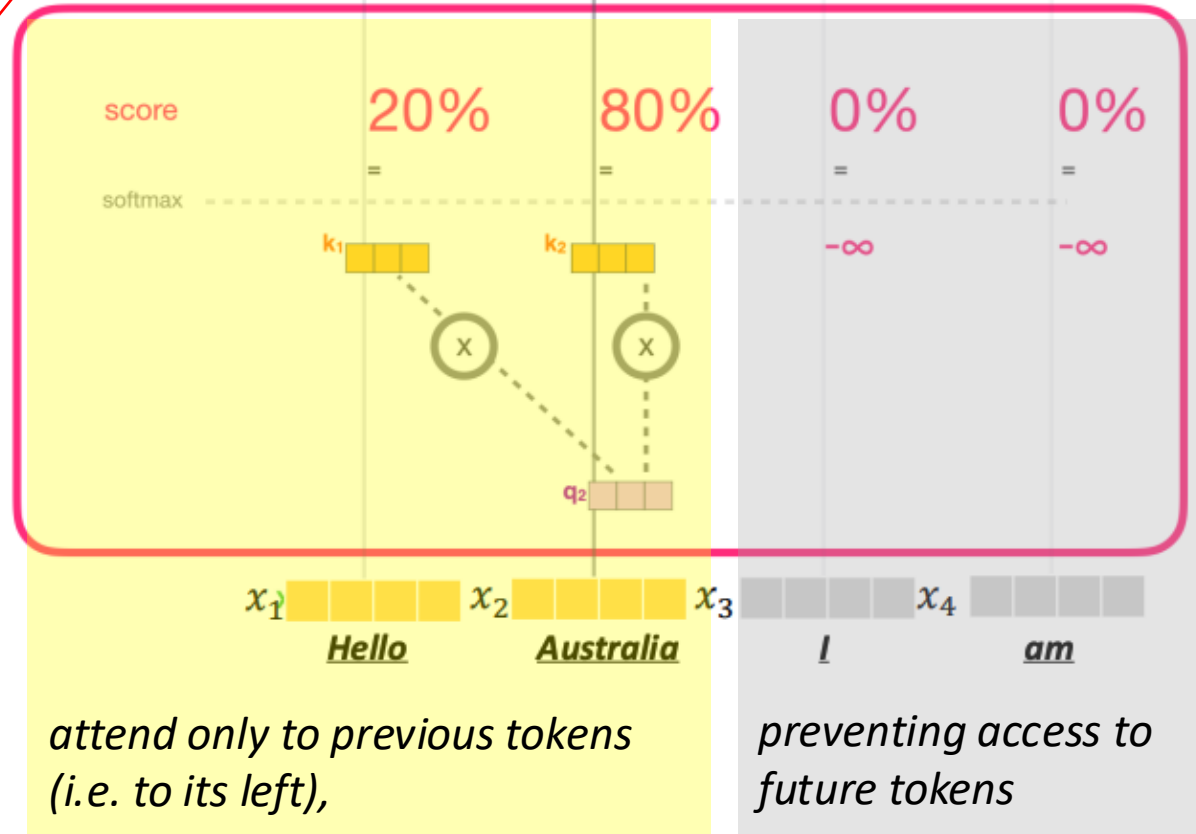
After token embedding: torch.Size([1, 20, 32])
After positional encoding: torch.Size([1, 20, 32])

Recap : Transformer (Masked Self-Attention)

Decoder-Only Block

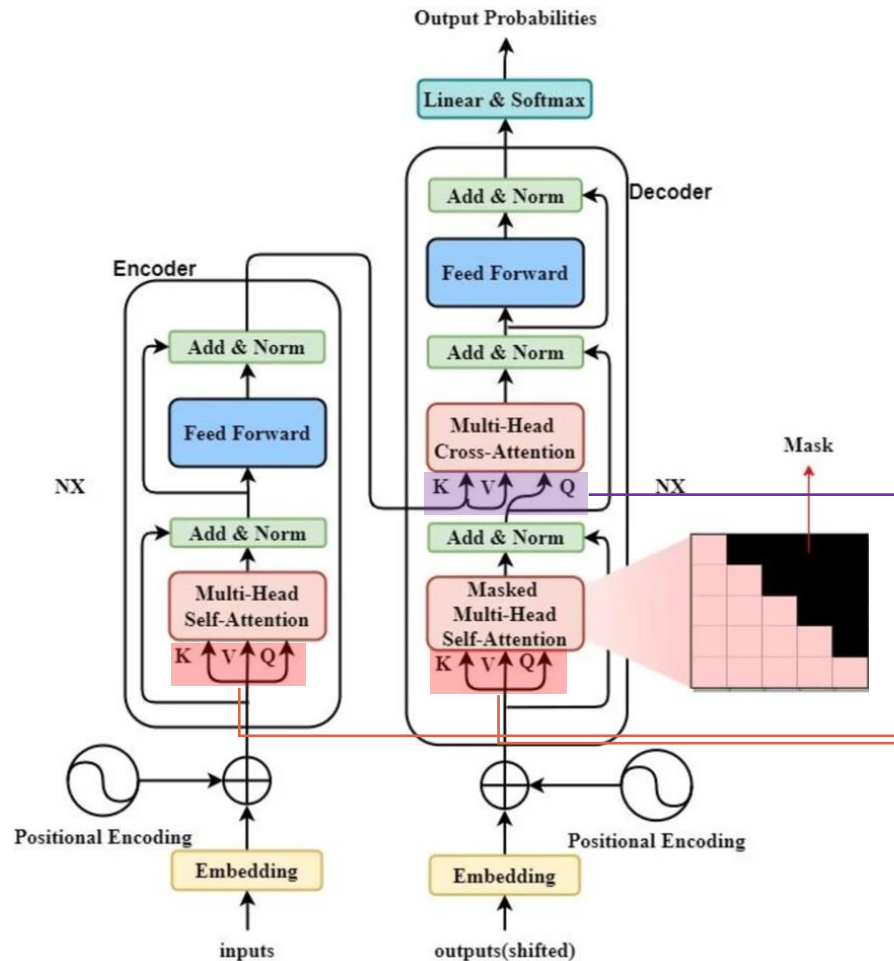


Masked Self-Attention



Recap : Transformer – FAQs

Q. Differences between attention, self-attention, and cross-attention?



Intuition

- **Attention** : look things up (most relevant parts)
- **Self-attention**: look within the same sentence
- **Cross-attention**: look at another sentence/input

Attention components:

- **Query (Q)**: what you're looking for
 - **Key (K)**: what you compare against
 - **Value (V)**: the information you retrieve
- Self-attention**: Q, K, and V all come from the same sequence (Captures relationships within the same sequence)
- Cross-attention**: Q comes from one sequence, but K and V come from a different sequence (Connects two different sequences)

Recap : Transformer – FAQs

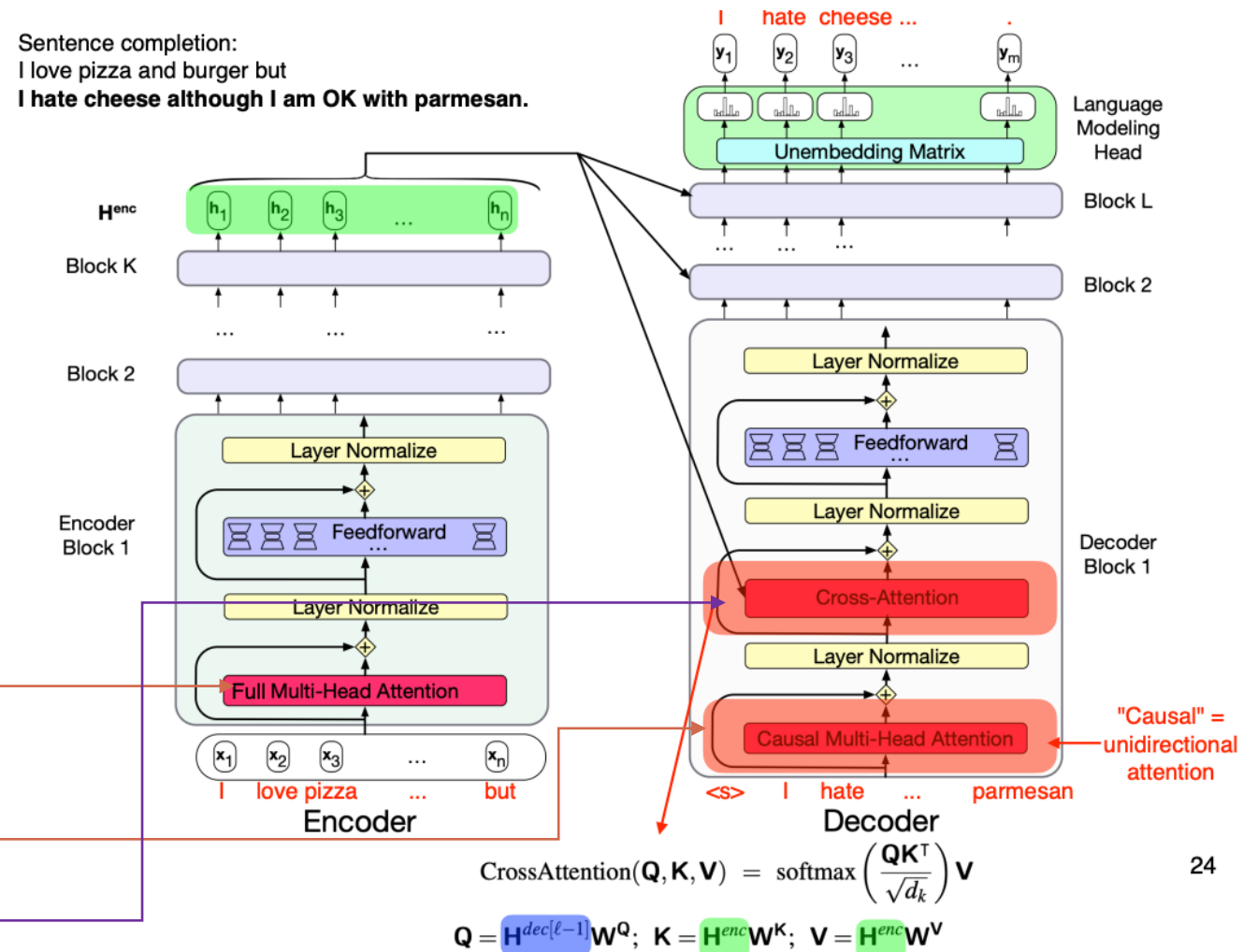
Q. What is the input in the Decoder?

Decoder-only (Week 6's code):

- **Decoder input:** All token sequence
- **Masked unidirectional self-attention** (causal masking: tokens attend only to previous tokens (i.e. to its left), preventing access to future tokens)

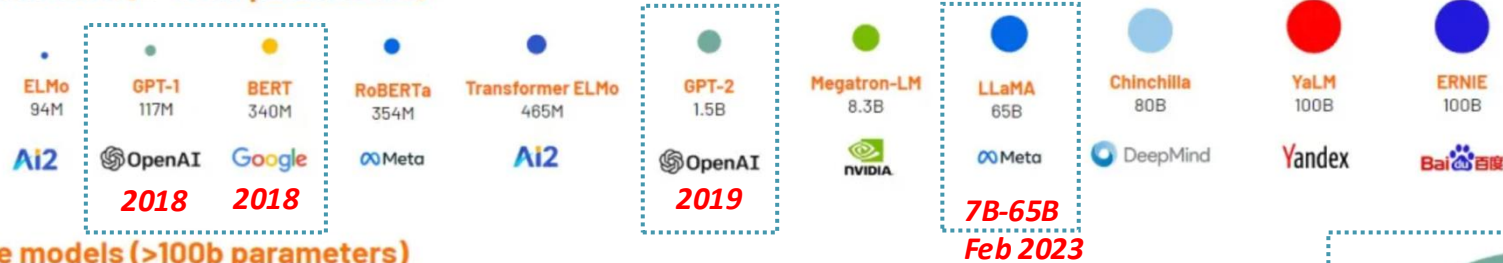
Encoder-Decoder:

- **Encoder input:** The full source sequence, all at once.
- **Bidirectional self-attention** — every token can attend to every other token.
- **Decoder input:** The target sequence generated, starting with <start>
- **Masked unidirectional self-attention**
- **Cross-attention** : how the decoder's queries (Q) attend to the encoder outputs (K, V)

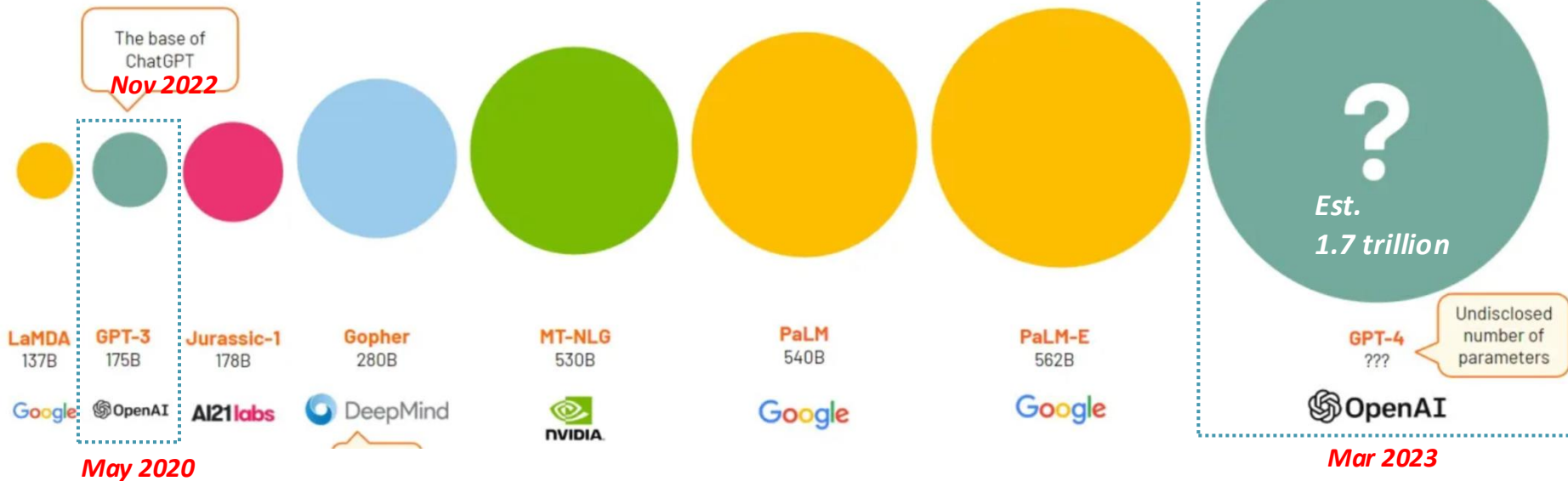


Large Language Models

Small models (<= 100b parameters)



Large models (>100b parameters)



- **Scale** : a billion to a trillion parameters (PLMs -> large parameter size)
- **Architecture** (Transformer Variants)

LLMs performance - Scaling Laws

- Scaling up language models improves the **model performance** -> “**What factors to be scaled?**”
- OpenAI team (Jan 2020) : **Model size (parameters), Dataset size, and Amount of compute** -> **lead to larger model** (e.g. GPT-3)
- DeepMind team (Mar 2022): Model size and the number of training tokens should be scaled equally. -> Chinchilla 70B outperforms.
- Human brain : Est. Average 86B neurons.

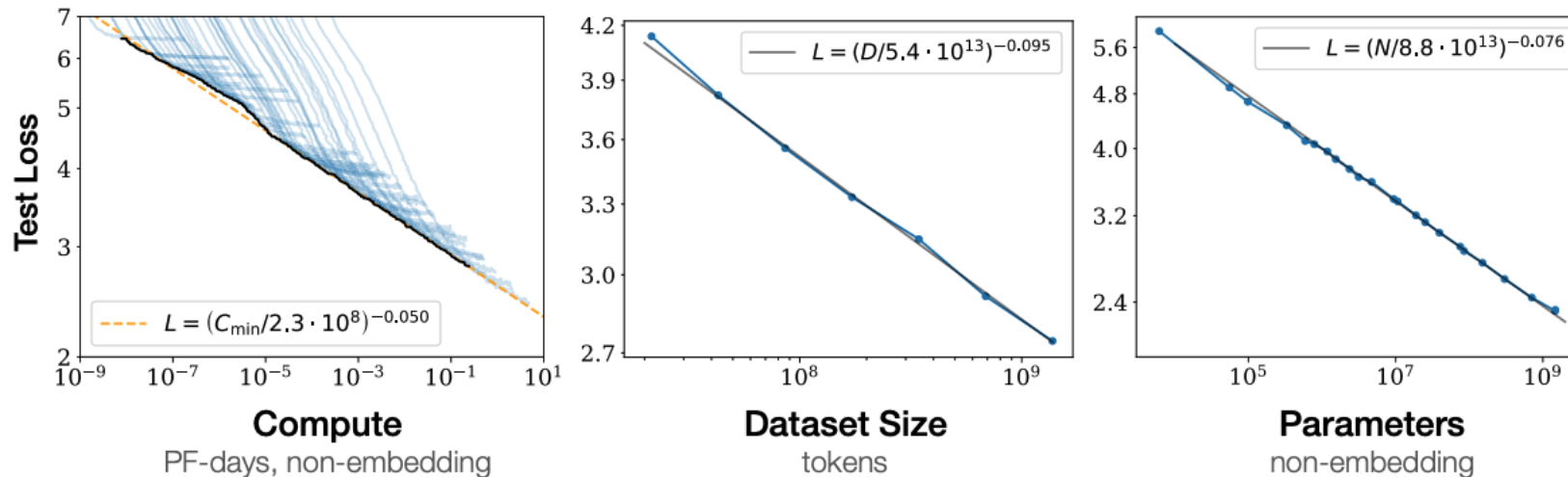


Fig : Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Evolutionary Trends of LLMs

Encoder-only (e.g. BERT)

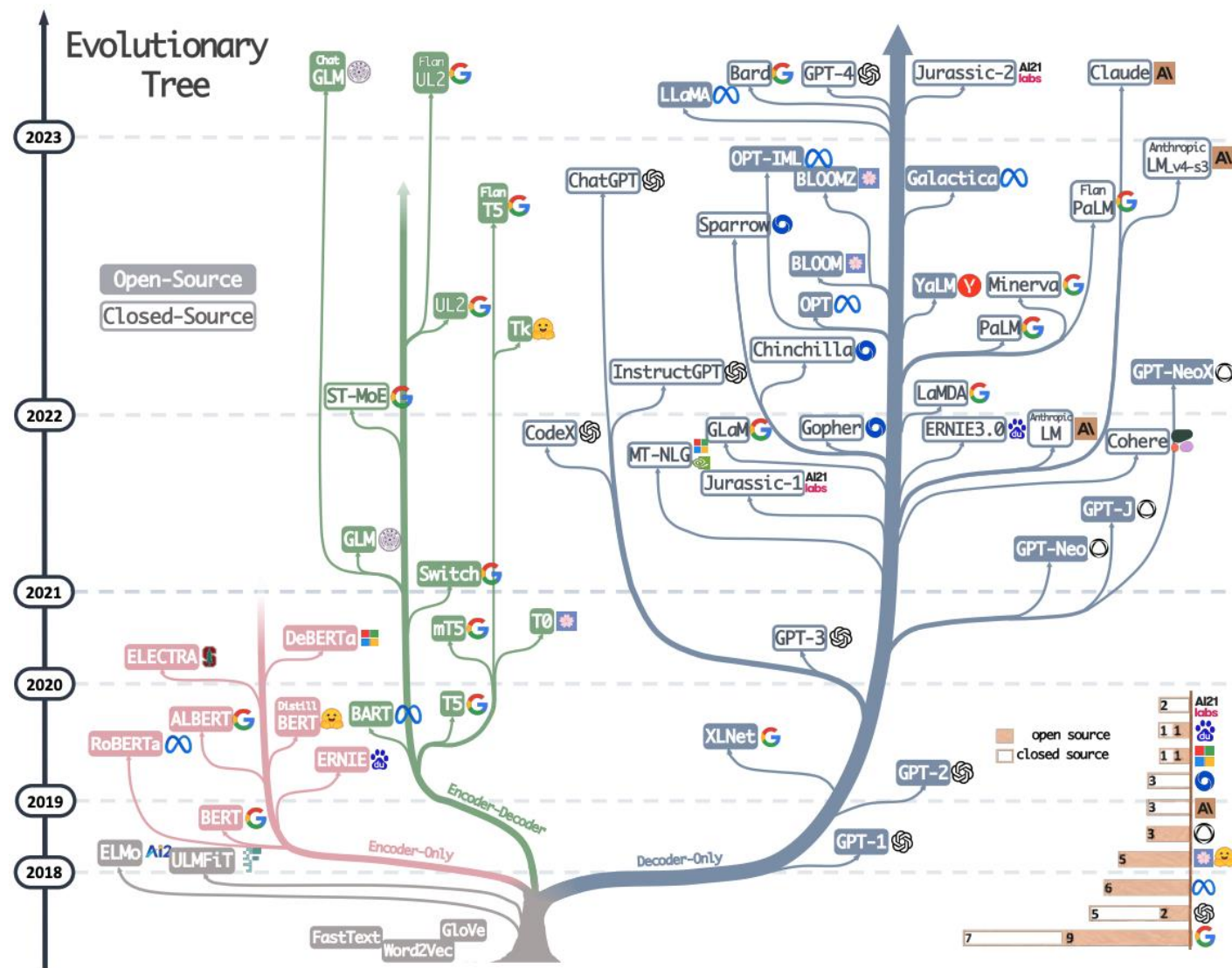
- Training : **Masked LMs**
- Task type : **Classification** (doesn't work for generation tasks)
- Pretrain task : **Predict masked words**
- Conditioned on **Bidirectional attention**

Decoder-only (e.g. GPT)

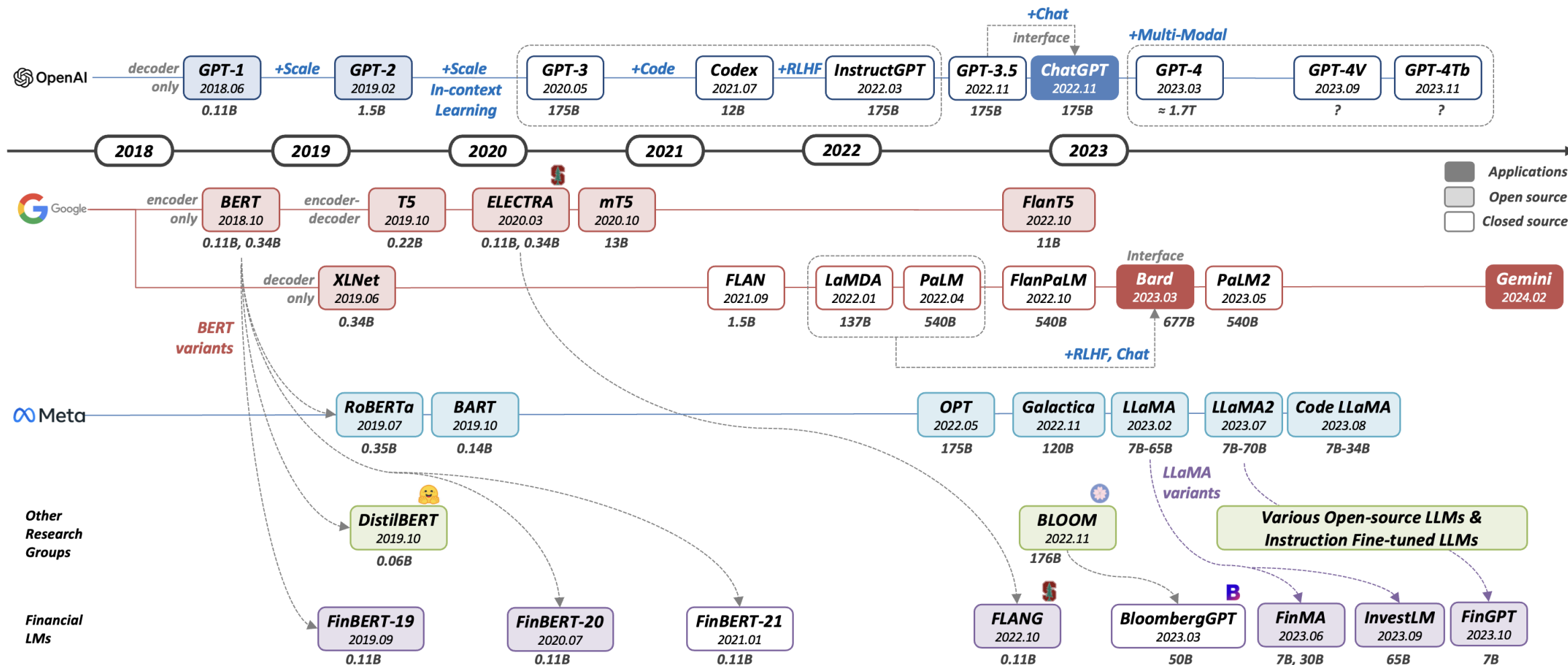
- Training : **Autoregressive LMs**
- Task type : **Generative and Classification**
- Pretrain task : **Predict next word**
- Conditioned on **Unidirectional attention**

Encoder-Decoder (e.g. T5)

- Task examples : Machine Translation, sentence completion



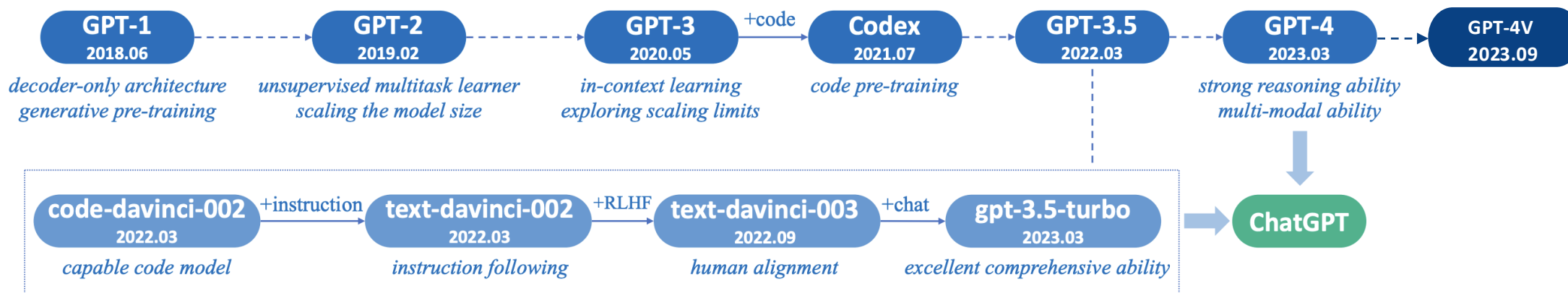
Trends of Domain-specific LLMs (e.g. FinLLMs)



Evolutionary Trends of GPT-series

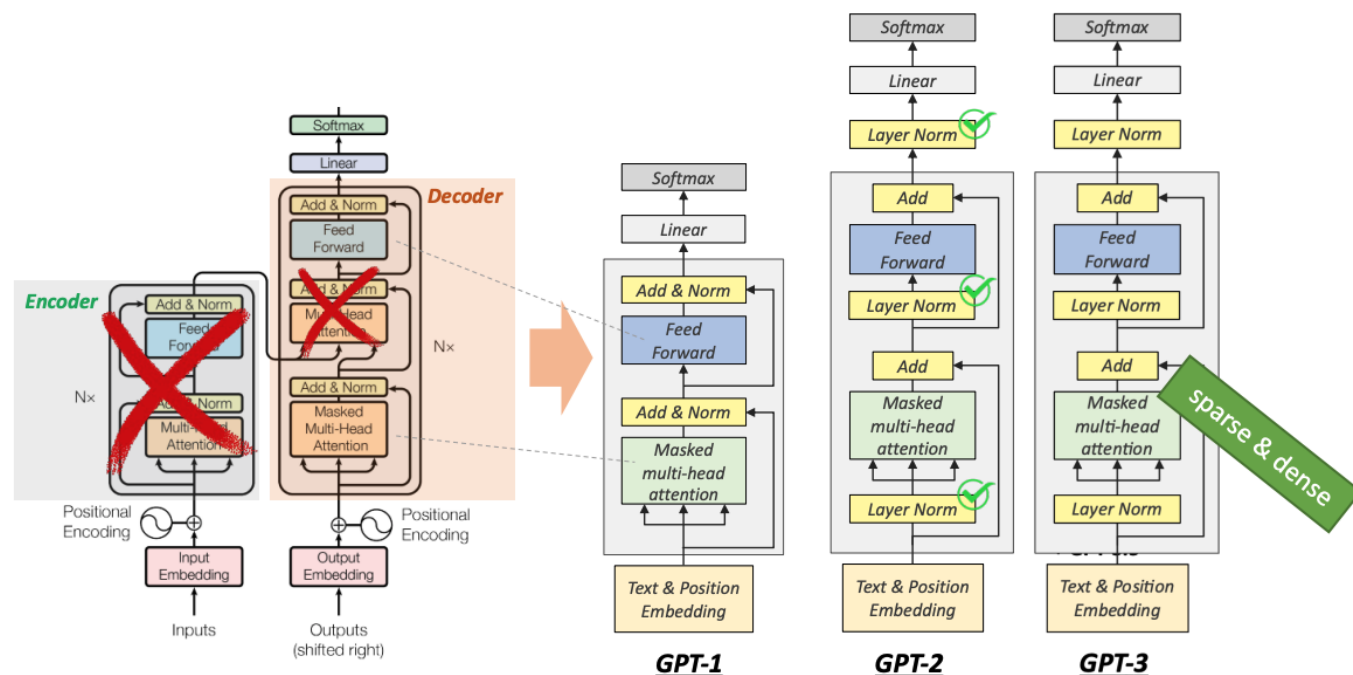
- **GPT1 (2018, 0.1B)** : Uni-directional Transformer (generative, decoder only), Supervised fine-tuning.
- **GPT2 (2019, 1.5B)** : Scale, zero-, a probabilistic form for multi-task solving, i.e., $p(\text{output}|\text{input}, \text{task})$
- **GPT3 (2020, 175B)** : LLM (scaling laws), In-context learning (zero-, one-, few-shot learning)
- **Codex (2021, 12B)** : Code dataset 159GB -> GitHub Copilot (12B), code-davinci-002 (175B)
- **InstructGPT (2022, 175B)** : human alignment (RLHF) | GPT3.5 + Instruct GPT + Codex -> chatGPT application
- **GPT4 (2023, Est. 1700B = 1.7T)** : human-level performance (e.g. passing bar exam), multi-modal ability

**Generative
task
Scaling**



- [GPT 1] Improving Language Understanding by Generative Pre-Training (A Radford et al., OpenAI, Jun 2018) [paper]
- [GPT 2] Language models are unsupervised multitask learners (A Radford et al., OpenAI, Feb 2019) [paper]
- [GPT 3] Language Models are Few-Shot Learners (TB Brown et al., OpenAI, May 2020, NIPS 2020) [paper]
- [Codex] Evaluating Large Language Models Trained on Code (M Chen et al, OpenAI, Jul 2021) [paper]
- [InstructGPT] Training language models to follow instructions with human feedback (L Ouyang et al., OpenAI, Mar 2022, NIPS 2022) [paper]
- [GPT 4] GPT-4 Technical Report (OpenAI, Mar 2023) [paper]
- Sparks of Artificial General Intelligence: Early experiments with GPT-4 (S Bubeck et al., Microsoft Research, Apr 2023) [paper]
- Fig : A Survey of Large Language Models (WX Zhao et al., Sep 2023) [paper] [github]

Discussion: GPT2 & GPT3



- Main differences of GPT2 and GPT3?

Lies in scale

- Model size
- Training Data
- Capabilities

	GPT-1	GPT-2	GPT-3
Training Data	BookCrawl	WebText	CommonCrawl
Parameters	117M	1.5B	175B
Layers	12	48	96
Dimensions	768	1600	12288

← Unsupervised Learning →

ChatGPT

+ GPT-3.5

*Supervised
Reinforcement
Learning*

Discussion: GPT2 & GPT3

• Main differences of GPT2 and GPT3?

Model Size

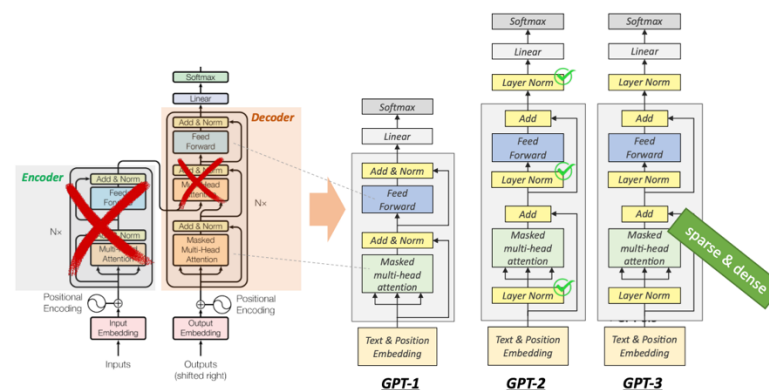
- GPT-2 has 1.5 billion parameters.
- GPT-3 massively scales up to 175 billion parameters—over 100x larger.

Training Data

- GPT-2 was trained on 40 GB of text data.
- GPT-3 was trained on 570 GB of filtered data (from a much larger corpus), enabling broader language understanding.

Capabilities

- GPT-2 had **limited generalization** in these tasks and often required fine-tuning.
- GPT-3 introduced **in-context learning** much more effectively: Zero-shot, one-shot, and few-shot learning are significantly more powerful in GPT-3 due to its **scale**.



	GPT-1	GPT-2	GPT-3	ChatGPT
Training Data	BookCrawl	WebText	CommonCrawl	+ GPT-3.5
Parameters	117M	1.5B	175B	Supervised Reinforcement Learning
Layers	12	48	96	
Dimensions	768	1600	12288	

← Unsupervised Learning →

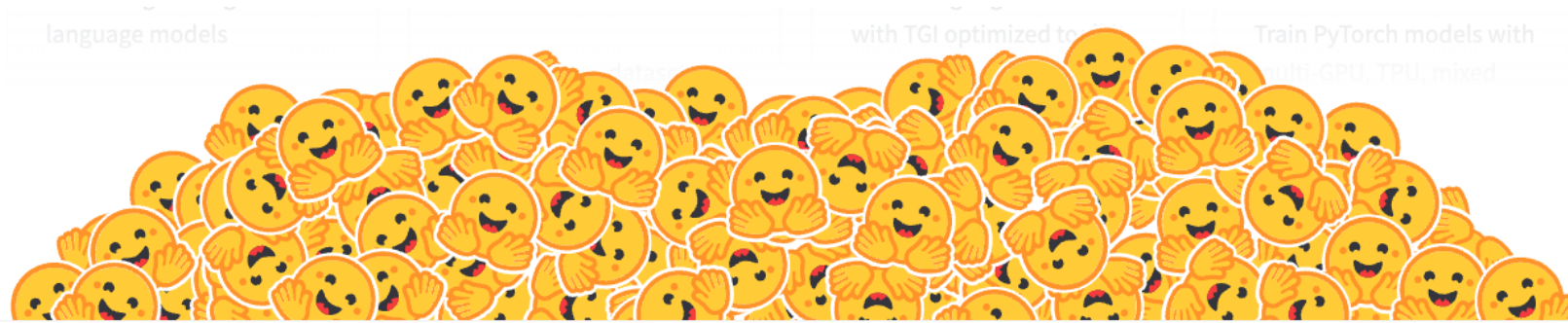
Hugging Face

GitHub for AI / Machine Learning

- A **platform and community** for AI and machine learning that enables users to discover, share, and collaborate on models and datasets.
- Known for its **Transformers Python library**, which simplifies downloading, training, and deploying advanced ML models.
- Offers tools for model hosting, fine-tuning, evaluation, and deployment, supporting tasks in NLP, computer vision, speech, and more.

Benefits of using Hugging Face

- **Standardization and Reproducibility**
- Centralized Repository
- Open-Source Collaboration
- Easy access
- Models and datasets freely available





(e.g.) What you can do with Hugging Face...

(e.g.) Dataset : Share it on GitHub and generate the Hugging Face datasets by adding scripts according to the HF guidelines.

The screenshot shows the Hugging Face interface for the dataset 'jeanlee/kmhas_korean_hate_speech'. The top navigation bar includes the Hugging Face logo, a search bar, and links to Models, Datasets, Spaces, Posts, Docs, Enterprise, and Pricing. The dataset page features a header with the dataset name, a 'like' button (20), and various filters for Tasks (Text Classification), Modalities (Text), Sub-tasks (multi-label-classification, hate-speech-detection), Languages (Korean), Size (100K - 1M), ArXiv (arxiv:2208.10684), Tags (K-MHaS, Korean NLP, Hate Speech Detection, Dataset, Coling2022), Libraries (Datasets, Croissant), and License (cc-by-sa-4.0). Below the header are tabs for Dataset card, Data Studio, Files and versions, Community (1), and Settings. The main content area is divided into two sections: the Dataset Viewer on the left and a sidebar on the right. The Dataset Viewer shows a split of 3 (train: 79k rows) and a search bar. It displays a table with columns 'text' (string) and 'label' (sequence). The 'text' column has a histogram showing a distribution from 1 to 305. The table contains several rows of Korean text and their corresponding labels. The sidebar on the right shows 'Downloads last month' (225), buttons for 'Use this dataset', 'Edit dataset card', and 'Papers with Code', and links to the homepage (K-MHaS), repository (Korean Multi-label Hate Speech Dataset), paper (K-MHaS: A Multi-label Hate Speech Detection Dataset i...), point of contact (Caren Han), sample code (Colab), size of downloaded dataset files (9.5 MB), size of the auto-converted Parquet files (7.28 MB), and number of rows (109,692).

Dataset Viewer

Split (3)
train · 79k rows

Search this dataset

text	label
"자한당들먹들.. 악플질 고만해라."	[2, 4]
정치적으로 편향된 평론한은 분은 별로...	[8]
적당히좀 쳐먹지. 그랬나??? 안그래도 문제인 때문에 나라 엉망진창인데...	[2]
"안서는 야재들 풀발기 ㅋㅋㅋ"	[4]
우와 ㅋㅋ 능력자	[8]
맛녀석 콩트보다 악했음맛녀석 애청자로써 70%실력발휘	[8]

Downloads last month 225

Use this dataset **Edit dataset card** **Papers with Code**

Homepage: K-MHaS **Repository:** Korean Multi-label Hate Speech Dataset

Paper: K-MHaS: A Multi-label Hate Speech Detection Dataset i...

Point of Contact: Caren Han **Sample code:** Colab

Size of downloaded dataset files: 9.5 MB

Size of the auto-converted Parquet files: 7.28 MB **Number of rows:** 109,692

K-MHaS: A Multi-label Hate Speech Detection Dataset in Korean Online News Comment (COLING 2022)

https://huggingface.co/datasets/jeanlee/kmhas_korean_hate_speech



Programming!

Large Language Models (LLMs) and NLP using libraries from the [Hugging Face](#) ecosystem.

1. **Understanding HuggingFace - Transformers, what can they do?**

We demonstrate various natural language processing (NLP) tasks using Hugging Face's transformers library.

2. **Processing the data**

We demonstrate how to prepare and process the dataset

3. **Fine-tuning a model**

How to use the high-level Trainer API to fine-tune a model using wandb (please make sure you create an account for [wandb](#) and retrieve an api key from <https://wandb.ai/authorize>)



Programming!

Hugging Face API Keys to Google Colab

Hugging Face (HF) API Tokens

- **Create an account on Hugging Face:** Go to Access Tokens [\[Link\]](#) → Create a new token (give it 'write' permission) → Copy and save your API key (if you forget it, that's okay — you can generate a new one).

User Access Tokens

+ Create new token

Access tokens authenticate your identity to the Hugging Face Hub and allow applications to perform actions based on token permissions. **Do not share your Access Tokens with anyone;** we regularly check for leaked Access Tokens and remove them immediately.

Name	Value	Last Refreshed Date	Last Used Date	Permissions
unimelb	hf_...cBvt	May 1, 2025	about 23 hours ago	WRITE

Google Colab to connect

- **In your Colab Notebook:** Click the 'key' icon on the left panel → Add a new secret → Allow notebook access → Name it 'HF_TOKEN' → Paste your Hugging Face API key as the value.

The screenshot shows the Google Colab interface. On the left sidebar, the 'Secrets' icon (a key) is highlighted with a red box. The main panel displays the 'Secrets' configuration. It includes a toggle for 'Notebook access' which is turned on, a text input for the name 'HF_TOKEN', and a masked input for the value. Below the inputs are icons for visibility, copy, and delete. At the bottom, there is a '+ Add new secret' button.



Programming!

W&B API Keys to Google Colab

- **Create an account on W&B: Go to API keys [Link] → Create a new token → Copy and save your API key**

API keys

Manage the API keys associated with your user account. API keys grant full access to your W&B account and should be kept secure.

2 keys

+ New key

Search by name or API key id

- **In your Colab Notebook: Run the code block → Follow the instruction**



```
1 import wandb
2
3 wandb.login()
```

```
... /usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
| |_| | '_ \ / _ \ | _/ -_)
wandb: (1) Create a W&B account
wandb: (2) Use an existing W&B account
wandb: (3) Don't visualize my results
wandb: Enter your choice: 2
wandb: You chose 'Use an existing W&B account'
wandb: Logging into https://api.wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: Create a new API key at: https://wandb.ai/authorize?ref=models
wandb: Store your API key securely and do not share it.
wandb: Paste your API key and hit enter: .....
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: jeanlee to https://api.wandb.ai. Use `wandb login --relogin` to force relogin
True
```